

Multimodal Large Model + Robotic Arm Gripping

Function Description

After the program runs, input the robotic arm gripping action command through the terminal. Based on the type of object to be gripped, the large model will start an external program to control the robotic arm to grip the target object.

Block height and width limits: both must not exceed 3 centimeters.

Startup

Open the terminal and enter the following command:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=True
```

Then open a second terminal and enter the following command:

```
ros2 run text_chat text_chat
```

The robotic arm gripping has the following types of action commands:

- **Grip by machine code ID:** Sort machine codes 1/2/3/4
- **Grip other objects:** Grip the small yellow duck on the desktop, grip the blue cuboid on the desktop

Gripping by Machine Code ID

Startup

In the text_chat terminal, input:

```
Sort machine code 2
```

```
root@yahboom:/# ros2 run text_chat text_chat
user input: Sort machine code 2
okay@, let me think for a moment... -[INFO] [1775116763.880772850] [text_chat_node]: "action": ['apriltag_sort(2)'], "response": Okay, I'll sort machine code 2 for you!
user input: [INFO] [1775116844.552161709] [text_chat_node]: "action": ['finishtask()'], "response": I've successfully sorted machine code 2! Let me know if you need help with anything else.
```

Then two terminals will open, respectively starting the machine code recognition program and the gripping program. After recognizing machine code 2, the gripping program will control the robotic arm to grip machine code 2 with the lower claw, then place it at the designated position, and finally return to the machine code recognition pose to prepare for the next gripping. Only after no longer detecting machine code 2 will it feedback to the large model that the task is complete.

Task Planning

Plan the action function `apriltag_sort(2)`, where the parameter 2 represents the ID of the machine code to be gripped.

Core Code Analysis

The apriltag_sort function source code is located at:

LargeModel_ws/src/largemodel/largemodel/action_service.py

```
def apriltag_sort(self, target_id): # Grip machine code
    #self.check_apriltag_sort()
    target_idf = float(target_id)
    #Start the gripping program
    cmd1 = "ros2 run largemodel_arm grasp"
    #Start the machine code recognition program, passing in the parameter target_id
    cmd2 = f"ros2 run largemodel_arm apriltag_sort --ros-args -p target_id:=
{target_idf:.1f}"
    subprocess.Popen(
        [
            "gnome-terminal",
            "--title=grasp_desktop_apritag",
            "--",
            "bash",
            "-c",
            f"{cmd1}; exec bash",
        ]
    )
    subprocess.Popen(
        [
            "gnome-terminal",
            "--title=apriltag_sort",
            "--",
            "bash",
            "-c",
            f"{cmd2}; exec bash",
        ]
    )
```

The grasp program source code path is: LargeModel_ws/src/largemodel_arm/largemodel_arm/grasp.py

```
#Create a subscriber, subscribe to the position information topic
self.sub = self.create_subscription(AprilTagInfo, 'PosInfo', self.pos_callback, 1)
#Position information topic callback function
def pos_callback(self, msg):
    pos_x = msg.x
    pos_y = msg.y
    pos_z = msg.z
    self.cur_tagId = msg.id
    if pos_z!=0.0:
        print("xyz id : ", pos_x, pos_y, pos_z, self.cur_tagId)
        #Start coordinate system transformation
        camera_location = self.pixel_to_camera_depth((pos_x, pos_y), pos_z)
        PoseEndMat = np.matmul(self.EndToCamMat, self.xyz_euler_to_mat(camera_location, (0,
0, 0)))
        EndPointMat = self.get_end_point_mat()
```

```

WorldPose = np.matmul(EndPointMat, PoseEndMat)
#Convert the rotation transformation matrix to xyz and Euler angles to get the
target pose
pose_T, pose_R = self.mat_to_xyz_euler(WorldPose)
pose_T[0] = pose_T[0] + self.x_offset
pose_T[1] = pose_T[1] + self.y_offset
pose_T[2] = pose_T[2] + self.z_offset
print("pose_T: ",pose_T)
if self.grasp_flag == True :
    print("Take it now.")
    self.grasp_flag = False
    #Pass the target position information to the grasp function for inverse
kinematics to get each angle value of the robotic arm
    threading.Thread(target=self.grasp, args=(pose_T,)).start()
#Call the inverse kinematics service, pass in the target position, and get the values of
servos 1-5.
def grasp(self,pose_T):
    print("-----")
    print("pose_T: ",pose_T)

    request = Kinemarics.Request()
    request.tar_x = pose_T[0]
    request.tar_y = pose_T[1] + 0.01
    request.tar_z = pose_T[2] + 0.01
    request.kin_name = "ik"
    request.roll = -1.0 #((2.5*request.tar_y*100)-207.5)*(math.pi / 180)

    # print("calcutelate_request: ",request)

    try:
        future = self.client.call_async(request)
        rclpy.spin_until_future_complete(self, future, timeout_sec=5.0)
        response = future.result()
        print("calcutelate_response: ",response)
        joints = [0.0, 0.0, 0.0, 0.0, 0.0,0.0]
        joints[0] = response.joint1 #response.joint1
        joints[1] = response.joint2
        joints[2] = response.joint3

```

The machine code recognition program apriltag_sort source code path is:

`LargeModel_ws/src/largemodel_arm/largemodel_arm/apriltag_sort.py`,

Gripping Other Objects

Startup

In the text_chat terminal, input:

Grip the red block on the desktop

```

root@yanboon:~/# ros2 run text_chat text_chat
user input: Grip the red block on the desktop
okay@, let me think for a moment... \[INFO] [1775117219.638365080] [text_chat_node]: Decision making AI planning:1. Call 'seewhat()' to observe the environment and get the outer frame coordinates (x1, y1, x2, y2) of the red block.
2. Call 'grasp_obj(x1, y1, x2, y2)' to grip the red block.
[INFO] [1775117222.447301130] [text_chat_node]: "action": ['seewhat()'], "response": "Okay, I'm going to grab the red block for you. Let me first take a look around and locate it."
[INFO] [1775117233.020752004] [text_chat_node]: "action": ['grasp_obj(380, 315, 475, 425)'], "response": "I found the red block! It's right there on the table. Now, I'm going to grab it for you."
[INFO] [1775117260.650575998] [text_chat_node]: "action": ['finishtask()'], "response": "I've successfully gripped the red block! Let me know what you'd like to do next."

```

The program will first take a photo to determine the position of the red block, then open two terminals to respectively start the KCF tracking positioning program and the object gripping program. After calculation, the robotic arm will lower the claw to grip the red block and finally return to the initial pose.

Task Planning

1. Call `seewhat()` to check the position of the red block;
2. Call `grasp_obj(x1,y1,x2,y2)` gripping function, where `x1,y1,x2,y2` represent the top-left and bottom-right corner coordinates of the red block's outer bounding box.

Core Code Analysis

The `seewhat()` function was analyzed in the previous tutorial [2. Multimodal Visual Understand]. It will not be repeated here. Let's mainly look at the `grasp_obj` function, source code located at:

`LargeModel_ws/src/largemodel/largemodel/action_service.py`

```

def grasp_obj(self, x1, y1, x2, y2):
    """
    Grip object
    x1,y1,x2,y2: Outer frame coordinates of the object
    """

    #Arm.Arm_serial_servo_write6(90,120,12,20,90,30,1000)
    #self.check_close_grasp_obj()
    #Object gripping program
    cmd1 = "ros2 run largemodel_arm KCF_Grap_Move"
    #KCF tracking positioning program
    cmd2 = "ros2 run largemodel_arm ALM_KCF_Tracker"
    # cmd3 = "ros2 run --prefix 'gdb -ex run --args' M3Pro_KCF ALM_KCF_Tracker_Node"
    subprocess.Popen(
        [
            "gnome-terminal",
            "--title=ALM_KCF_Tracker",
            "--",
            "bash",
            "-c",
            f"{cmd2}; exec bash",
        ]
    )
    time.sleep(5.0) #wait for ALM_KCF_Tracker to start
    subprocess.Popen(
        [
            "gnome-terminal",
            "--title=grasp_desktop",
            "--",
            "bash",
            "-c",
            f"{cmd1}; exec bash",
        ]
    )

```

```

time.sleep(2.0)
if self.stack_flag == True:
    self.get_logger().info('Publish the stack_step topic...')
    step_ = Int16()
    step_.data = self.step
    self.step_pub.publish(step_)
    self.step = self.step + 1
    #Publish outer frame coordinate information topic data, which will be subscribed to by
the ALM_KCF_Tracker node
    x1 = int(x1)
    y1 = int(y1)
    x2 = int(x2)
    y2 = int(y2)
    self.object_position_pub.publish(Int16MultiArray(data=[x1, y1, x2, y2]))

```

The KCF_Grap_Move program source code location:

LargeModel_ws/src/largemodel_arm/largemodel_arm/KCF_Grap_Move.py

```

#Import library, the path of this library is
LargeModel_ws/src/largemodel_arm/largemodel_arm/Dofbot_Track.py
from largemodel_arm.Dofbot_Track import *

#Create robotic arm tracking and gripping object
self.dofbot_tracker = DofbotTrack()

#Create subscriber, subscribe to the /pos_xyz topic
self.pos_sub = Subscriber(self,Position,'/pos_xyz')

#Callback function
def TrackAndGrap(self,position):
    center_x, center_y = position.x,position.y
    self.cur_distance = position.z
    if self.done == True:
        self.get_current_end_pos()
        self.get_logger().info('-*-*-*-*-*-*-*-*')
        print("self.CurEndPos: ",self.CurEndPos)
        #Call external library function Clamping
        self.dofbot_tracker.Clamping(center_x,center_y,self.cur_distance,self.CurEndPos)
        self.done = False

#View the Clamping function in Dofbot_Track.py
def Clamping(self,cx,cy,cz,CurEndPos):
    self.CurEndPos = CurEndPos
    #Start coordinate system transformation
    camera_location = self.pixel_to_camera_depth((cx,cy),cz)
    print("cz: ",cz)
    PoseEndMat = np.matmul(self.EndToCamMat, self.xyz_euler_to_mat(camera_location, (0, 0,
0)))
    EndPointMat = self.get_end_point_mat()
    WorldPose = np.matmul(EndPointMat, PoseEndMat)
    #Modify the rotation transformation matrix to xyz and Euler angles
    pose_T, pose_R = self.mat_to_xyz_euler(WorldPose)

```

```

pose_T[0] = pose_T[0] + self.x_offset
pose_T[1] = pose_T[1] + self.y_offset
pose_T[2] = pose_T[2] + self.z_offset
print("pose_T: ",pose_T)
#Create inverse kinematics service object, and assign the calculated pose_T representing
the target position
request = Kinemarics.Request()
if pose_T[0]>0:
    request.tar_x = pose_T[0] +0.01
else:
    request.tar_x = pose_T[0] - 0.01
request.tar_z = pose_T[2]
request.tar_y = pose_T[1] + request.tar_z*0.2
request.kin_name = "ik"
request.roll = -1.0
print("calcutelate_request: ",request)

try:
    #Call inverse kinematics service to calculate the values of servos 1-4
    future = self.client.call_async(request)
    rclpy.spin_until_future_complete(self, future, timeout_sec=10.0)
    response = future.result()
    joints = [0.0, 0.0, 0.0, 0.0, 0.0,0.0]
    joints[0] = response.joint1 #response.joint1
    joints[1] = response.joint2
    joints[2] = response.joint3
    if response.joint4>90:
        joints[3] = 90.0
    else:
        joints[3] = response.joint4
    joints[4] = 90.0
    joints[5] = 20.0
    print("compute_joints: ",joints)

```

The ALM_KCF_Tracker program source code location:

LargeModel_ws/src/largemodel_arm/largemodel_arm/ALM_KCF_Tracker.py

```

#Create publisher to publish object position information
self.pub_pos = self.create_publisher(Position, "/pos_xyz", 10)
#Create subscriber to subscribe to object outer frame information
self.xy_subscription =
self.create_subscription(Int16MultiArray,'corner_xy',self.GetXYCallback,qos_profile=1)

#Callback function
def GetXYCallback(self,msg):
    print("msg: ",msg.data)
    print(msg.data[0])
    print(msg.data[1])
    print(msg.data[2])
    print(msg.data[3])
    #Assign the topic message data to self.Roi_init
    self.Roi_init = (msg.data[0],msg.data[1],msg.data[2],msg.data[3])

```

```

self.Track_state = 'identify'
self.get_xy = True
self.rect = [msg.data[0],msg.data[1],msg.data[2],msg.data[3]]

#In the process function, based on the value of self.Roi_init, perform KCF initialization,
if self.gTracker_state == True:
    Roi = (self.Roi_init[0], self.Roi_init[1], self.Roi_init[2] - self.Roi_init[0],
self.Roi_init[3] - self.Roi_init[1])
    self.compute_roi = True
    print("Roi: ",Roi)
    self.gTracker = Tracker(tracker_type=self.tracker_type)
    self.gTracker.initworking(rgb_img, Roi)
    self.gTracker_state = False
#KCF tracking, get the object center point information,
rgb_img, (targBegin_x, targBegin_y), (targEnd_x, targEnd_y) = self.gTracker.track(rgb_img)
center_x = targEnd_x / 2 + targBegin_x / 2
center_y = targEnd_y / 2 + targBegin_y / 2
self.cx = center_x
self.cy = center_y
#In the kcfTrack image callback function,
#Assign the calculated center point coordinates to center_x, center_y
center_x, center_y = self.cx,self.cy
cv2.circle(depth_to_color_image,(int(center_x),int(center_y)),1,(255,255,255),10)
#Calculate the depth distance information of the center point
self.cur_distance = depth_image_info[int(center_y),int(center_x)]/1000.0
dist = round(self.cur_distance,3)
dist = 'dist: ' + str(dist) + 'm'
cv.putText(result_frame, dist, (30, 30), cv.FONT_HERSHEY_SIMPLEX, 1.0, (255, 0, 0), 2)
pos = Position()
pos.x = center_x
pos.y = center_y
pos.z = self.cur_distance
Track_pos = Position()
Track_pos.x = center_x
Track_pos.y = center_y
#Publish position information topic data
self.pub_track_pos.publish(Track_pos)

```